Monkey See, Monkey Prototype

A prototype makes an idea visible before it deserves to be trusted. This guide follows the useful fakes, rapid models, generated apps, staged demonstrations, and hidden workers that complicate the route from “possible” to “ready.”

Reading the title as a warning about imitation

The title rewrites “monkey see, monkey do,” a proverb about copying an action without necessarily understanding it. Prototyping can look similar from the outside. A team imitates the shape of a future product, but the version may not contain the machinery, safety, reliability, or organisation that the real product will require. The imitation is valuable when everyone knows which question it is testing. It becomes deceptive when appearance is used as proof of a capability that does not exist.

This section therefore needs more than a simple sequence from rough to polished. A paper interface can teach designers more than a beautiful coded screen because users will criticise paper freely. A 3D-printed shell can reveal whether a handle fits the hand without proving that the final material will survive heat. A chatbot can generate a working demonstration in minutes while leaving security and maintenance problems for later. A person hidden behind an “automated” service can help test demand, yet also expose private information and create a false claim about labour.

The official links are not a bonus gallery. They supply the evidence needed to separate these cases. Students should inspect the early iPhone hardware, follow the prototyping cycle, understand Sarah Boone’s practical invention, read the warnings attached to vibe coding, see the operator behind the Mechanical Turk, and compare public demonstrations with later reporting about teleoperation. The recurring question is not “Is it fake?” It is “What is simulated, who knows, what is learned, and what decision will be made before real users carry the risk?”

Chapter 1 - From rough draft to real product

A prototype is an early representation built to answer a question about a possible product or service. That broad definition includes sketches, cardboard forms, clickable screens, coded experiments, and near-production units. The prototype's quality is not measured by resemblance to the final object alone. It is measured by whether the representation is faithful enough to answer the current question without wasting effort on details that do not matter yet.

Start with the uncertainty, then choose the model

If a team is unsure whether people understand the order of screens in an app, a paper prototype may be enough. A tester points to a drawn button, and a researcher replaces the sheet to simulate the next screen. No code is required because the uncertain part is navigation. If the uncertainty concerns whether a hinge will bear weight, paper is the wrong medium. The team needs dimensions, materials, forces, and perhaps a physical proof of concept.

A sketch explores form quickly. Storyboarding shows a sequence of actions and the context around a product. A wireframe establishes information hierarchy without committing to visual finish. A mockup shows a more detailed appearance but may not work. A proof of concept demonstrates that a difficult mechanism is feasible, often without presenting a usable product. User testing observes people attempting representative tasks. Each term names a different answer to "What do we need to learn next?"

Low fidelity keeps the model cheap and easy to change. It can encourage honest criticism because nobody mistakes it for a finished decision. High fidelity becomes useful when timing, realistic content, detailed interaction, manufacturing fit, or emotional response matters. High fidelity too early creates attachment. Teams may protect polished work instead of revising the underlying idea.

The early object may look nothing like the product

The curriculum's gallery of famous prototypes (L086) makes a strong visual point: an early test rig often exposes components, cables, controls, and supporting equipment that disappear from the consumer version. That difference is not evidence of failure. Engineers deliberately build access around the part under investigation.

The linked iPhone M68 prototype (L087) resembles a circuit-board workbench more than a pocket phone. A development board allowed engineers to connect displays, radios, controls, and debugging tools while keeping the new device's industrial design secret. The visible bulk belonged to testing infrastructure, not the intended user experience. A scholar who judges it only by appearance misses its function.

This also explains why a prototype and a draft overlap without being identical. A written draft is an instance of the work itself: the sentences may survive into publication. A prototype can be a separate instrument used to learn about a future work. A foam model of a phone may contribute no material to the manufactured device, yet reveal where fingers reach. Both are provisional, but they preserve different kinds of evidence.

A cycle, not a straight staircase

The official prototyping cycle (L088) moves through concept, representation, testing, learning, and revision. Real projects may loop through those stages many times and use several prototypes at once. A team can test size with foam, software flow with a clickable screen, and manufacturing risk with a separate material sample.

Feedback is evidence only when the test matches the intended use. Asking friends whether they like a rendered image does not show that customers can assemble the product. Watching a tester complete a task while a designer explains every step hides usability problems. A good test defines the user, task, condition, and measure before the demonstration begins.

Failure is informative when it changes a decision. If the same prototype is repeatedly shown without modifying the design or test, the team may be performing iteration rather than learning. Document the assumption, observation, and next choice. This makes the unfinished route traceable.

MVP is not a synonym for careless launch

A minimum viable product is the smallest real offering that can test a significant business or user assumption. “Minimum” reduces unnecessary build. “Viable” means the version still provides enough value and safety for the people asked to use it. A minimum marketable feature is usually a smaller releasable unit that customers can understand and value. The terms are used inconsistently, so scholars should explain the function rather than recite a label.

An internal mockup may break without harming anyone. A public health service, payment system, or children’s product has a higher threshold because users can suffer from error. The ethical amount of completeness depends on exposure. Moving fast is not an excuse to transfer unknown risks to people who were told they were receiving a dependable product.

Transfer beyond gadgets

Schools can prototype a timetable on one year group, mark a proposed room layout with tape, or run a short version of an event before committing a full budget. Governments can use temporary street materials to test a crossing. Writers can table-read a scene. The principle is the same: make an assumption encounter reality while revision is still affordable.

The method fails when a pilot quietly becomes permanent without evaluation, when participants are not told they are testing something, or when a temporary result is generalised to a different population. Prototyping is not automatically humble. It becomes humble when the maker accepts that the model was built to be contradicted.

> **Sources for this chapter:** all three official product links: the gallery of early gadget prototypes (L086), the iPhone M68 development hardware (L087), and the linked prototyping cycle (L088). The chapter uses them to distinguish a test instrument from a rough final product.

Chapter 2 - Fast models, real risks

Digital simulation, computer-aided design, and additive manufacturing have shortened the distance between an idea and a physical object. A student can alter a file in the morning and hold a new geometry in the afternoon. That speed changes who can prototype and how many alternatives can be explored. It does not eliminate engineering, manufacturing, or responsibility.

Rapid means faster feedback, not instant truth

The curriculum's rapid prototyping overview (L090) describes methods for creating models directly from digital designs. A 3D printer builds an object layer by layer, which makes complex and customised forms possible without first making expensive production tooling. Designers can test fit, scale, assembly, and ergonomics before committing to a factory run.

A print is still a model of selected properties. A plastic part may match final geometry while behaving differently under load, sunlight, moisture, or heat. Layer orientation affects strength. Surface finish and manufacturing tolerances can differ from injection moulding or machining. If a team uses the prototype to answer a material question it cannot represent, speed produces false confidence.

Simulation has the same limit. A model predicts according to its equations, parameters, and input data. It is powerful when assumptions are validated and conditions are understood. It can miss unusual users, rare failures, or interactions the modeller did not include. Physical tests and real-world observation remain necessary when the consequences justify them.

A useful test plan separates properties explicitly. A printed inhaler shell can test grip and button reach without ever delivering medicine. A digital airflow model can compare internal geometry without proving that a patient will use the device correctly. A production-material sample can test chemical compatibility without answering whether instructions are understood. Only the combined evidence approaches readiness. "The prototype worked" is meaningless until the speaker names which property worked, for whom, and under what condition.

Sarah Boone started from an ordinary difficulty

The prompt pairs Thomas Edison with Sarah Boone (L089), an important correction to the habit of imagining invention only through famous laboratories. Boone, a dressmaker who was born into slavery and later became one of the early African American women to receive a United States patent, patented an improvement to the ironing board in 1892. Her narrow, curved, reversible board made it easier to iron sleeves and fitted garments.

The invention demonstrates problem fit. A flat table could press cloth, but it did not match the shape of the work. Boone's design embodied knowledge from repeated practice. Prototyping is often strongest when the maker understands a specific user and task rather than beginning with a technology and searching for a need.

Her example also complicates stories of individual genius. Access to education, patents, capital, manufacturing, and markets affects whose model becomes a recognised product. Makerspaces can widen access to tools, but a printer alone does not supply free time, mentorship, safe materials, legal support, or a customer network.

What a responsible makerspace needs

A school makerspace should match access to risk rather than restrict creativity through vague fear. Hand tools, lasers, soldering stations, and 3D printers have different hazards. Training can be staged: an orientation, a demonstrated skill, supervised use, and independent permission for lower-risk tasks. Clear ventilation, material lists, guards, maintenance, emergency procedures, and adult supervision matter more than a sign that says "be careful."

The curriculum packet adds the US Occupational Safety and Health Administration's 3D-printing guidance, which discusses potential exposure to particles, gases, hot surfaces, moving components, and finishing chemicals. Risk depends on the process and material. Responsible access means understanding those differences, not treating every printer as either harmless or forbidden.

Digital files create other risks. A model may reproduce a patented part, a restricted object, or someone's scanned body without consent. A design downloaded online may contain a structural weakness invisible in the preview. Tool access therefore needs an ethical conversation alongside physical safety.

Democratisation can still reproduce inequality

Rapid tools lower some barriers. A small team can test a shape without negotiating an overseas factory sample, and a student can learn by making. Yet commercial printers, reliable materials, maintenance, software, and expert time cost money. Well-resourced schools can iterate more often. Designers whose bodies and cultures already dominate reference data may continue to receive the best fit.

Inclusive prototyping deliberately recruits varied testers and pays attention to outliers. An adjustable handle should be tried by more than an average adult hand. An assistive device should be designed with disabled users rather than merely for them. An educational product should be tested under the connectivity and language conditions of its intended classrooms.

When to slow the cycle down

Early in exploration, many cheap failures are useful. As a product approaches real use, the test burden increases. Teams need traceable requirements, repeatable measurements, safety margins, and independent review. In regulated fields, validation asks whether the final product meets user needs while verification asks whether it meets specified design requirements. The exact vocabulary varies, but the direction is clear: confidence must become more demanding as exposure grows.

A good restriction is specific, proportionate, and teachable. "Only responsible people may use this" leaves authority vague. "Complete ventilation training, use approved materials, and obtain supervision for this machine" creates a path to access. The destination is not a locked workshop. It is a community capable of making without pretending that enthusiasm cancels consequence.

Rapid fabrication can also shorten the route to harmful objects, counterfeit replacement parts, or devices that look certified but are not. The answer is not to treat every file as dangerous. Makerspaces can classify high-risk categories, require provenance for safety-critical designs, prohibit claims that an untested part meets a standard, and teach users to document revisions. Responsibility becomes a design practice embedded in the workflow instead of a personality test applied at the door.

> **Sources for this chapter:** the official inventor profile of Sarah Boone (L089) and the official rapid-prototyping introduction (L090), supplemented by OSHA's 3D-printing safety guidance. These sources support access with defined safeguards rather than a general ban.

Chapter 3 - Vibe coding and the flood of new software

Vibe coding describes a style of software creation in which a person explains a desired program in natural language, accepts code produced by an AI system, runs it, and asks for revisions. The term became popular in 2025 and by 2026 covers everything from playful experiments to products built with AI assistance. That range matters. Using a model to explain one function is not the same as deploying an application whose maker cannot inspect any of it.

The interface hides the stack

The official IBM explanation of vibe coding (L091) describes how natural-language instructions can produce and refine software. The immediate gain is enormous. A beginner can make a working interface, automate a personal task, or test a product idea without first mastering syntax, package configuration, and deployment. Experienced programmers can offload routine scaffolding and explore alternatives quickly.

Yet a simple prompt may create a complicated dependency stack. The generated app can include libraries, authentication, database queries, cloud services, and code copied from patterns the user does not recognise. The interface looks small because the platform hides infrastructure. Someone still has to understand data flow, permissions, cost, licences, failure, and update paths.

Natural language is also ambiguous. “Make login secure” does not define password storage, rate limits, session expiry, recovery, multi-factor authentication, or threat assumptions. The model may produce a familiar pattern that appears to work in a demonstration while failing under attack. A prompt is not a specification merely because it is written in English.

Prototype speed can create production debt

For a disposable prototype, generated code can be entirely appropriate. It lets a team test whether users want the workflow before investing in robust architecture. The danger begins when the demonstration attracts real users and becomes the production system without a deliberate transition. Temporary shortcuts now store personal data and depend on packages nobody has reviewed.

Technical debt is not simply ugly code. It is future work created by an earlier decision. Some debt is rational when speed has clear value and the repayment plan is understood. Unknown debt is different. If the team cannot identify what was borrowed, it cannot estimate the cost of change.

The curriculum’s official concern about uncertain quality (L092) should be treated as a viewpoint, not a settled measurement of every generated program. The stronger claim is narrower: software that nobody understands is difficult to test, secure, maintain, and support. Quality depends on review practices, domain risk, user scale, and whether the builders can take responsibility after the first successful run.

Generated code also has provenance risk. A model may suggest a package that does not exist, an outdated method, or a dependency whose name resembles a malicious package. It may produce code that passes the happy-path demonstration while exposing an administrative function or sending data to the wrong service. Because the output is fluent, a novice may read confidence as verification. Every dependency should be resolved from its authoritative registry, pinned deliberately, scanned, and understood well enough that the team knows why it is present.

What testing has to cover

A generated application needs the same questions as hand-written software. Does it behave correctly for expected inputs? What happens at boundaries and under failure? Can one user access another user’s information? Are secrets stored safely? Do dependencies have known vulnerabilities or incompatible licences? Can the system be observed, backed up, restored, and updated? Is it accessible to keyboard and assistive-technology users?

Automated tests are useful only if they test the right behaviour. Asking the same model to generate the application and approve its own tests can reproduce one mistaken assumption twice. Human review, independent tools, user testing, and security analysis provide different forms of evidence. High-risk systems need stronger independence.

Maintenance begins before launch. Name an owner, document the architecture, record dependencies, and decide how reports will be handled. If the prototype has no path for support, limit its audience and lifetime. A link that works today is not automatically a service.

A practical validation ladder begins locally with synthetic data, then moves to a small invited test, a monitored beta, and wider release only after observed failures have owners. At each step, increase the realism of data and load while tightening logging, access control, backup, and incident response. The gate is evidence, not the number of prompts the maker has typed. A generated app that cannot explain its own data boundaries should not cross into a setting where those boundaries matter.

Should it be harder to get there?

Making software creation more accessible is generally valuable. People who understand a local problem can build a useful tool without waiting for a large vendor. Students can move from consuming software to questioning and shaping it. Disabled users can customise workflows that commercial products ignore.

The alternative to open access should not be a gate controlled only by professional status. Skilled programmers also create insecure systems. Better gates are attached to consequences: a personal offline toy needs little oversight; a public medical, financial, educational, or identity service needs review, privacy protection, and accountable operators. Distribution, not creation, is often the meaningful threshold.

A practical route from vibe to reliable software

Begin with a bounded problem and data that is non-sensitive or synthetic. Ask the system to explain the generated architecture and assumptions. Keep changes under version control so a working state can be recovered. Test one behaviour at a time, then invite a knowledgeable reviewer to challenge security and maintainability. Replace unexplained components before broad release. Document what the tool can and cannot do in language users can understand.

This route preserves the creative benefit of a fast prototype while refusing the fantasy that code becomes self-supporting when it appears on screen. Vibe coding changes who can begin the journey. It does not abolish the obligations attached to arrival.

> **Sources for this chapter:** the official IBM overview (L091) and the curriculum's linked critique of quality and maintenance risk (L092). The packet also points to security mitigations and an AI-assisted building platform; these are examples, not proof that all tools have the same risk.

Chapter 4 - The Mechanical Turk and the ethics of useful illusion

In 1770 Wolfgang von Kempelen presented a cabinet, chessboard, and turbaned mechanical figure that appeared able to play strong chess. The "Turk" toured for decades under different owners and defeated or impressed famous observers. A skilled human chess player was concealed inside the cabinet. The machine did not merely fail to explain its operation; its presentation actively redirected attention away from the operator.

A fake machine that asked a real question

The official history of the Mechanical Turk (L093) shows why the hoax matters to the history of artificial intelligence. Audiences already wanted to know whether reasoning could be mechanised. The performance made that future feel present, even though the impressive intelligence belonged to a hidden person.

The apparatus depended on more than a secret compartment. Its operator needed chess skill, a way to follow the board, controls for the figure's arm, and procedures for remaining concealed while sections of the cabinet were opened. The deception was engineered. That is precisely why the case transfers well to modern demos: a convincing front end can redirect attention from the labour and infrastructure behind it.

Amazon later named its microtask platform Mechanical Turk, making the historical comparison explicit. Online requesters can distribute small tasks to human workers, often in ways that support data labelling and AI systems. The worker is no longer physically inside a cabinet, but human intelligence can still disappear behind a technical interface and low unit price.

The historical costume matters too. The turbaned figure turned an imagined "Oriental" other into spectacle for European audiences. The cabinet hid a person physically while the character helped audiences accept mystery as exotic machinery. Modern product language can perform a related erasure when it describes distributed workers as a neutral cloud or a tiny piece of "human-in-the-loop" infrastructure. A responsible account names who performs the work, where decisions occur, and what conditions make the apparent automation possible.

Wizard of Oz testing changes the ethical conditions

In Wizard of Oz testing (L094), users interact with what appears to be an automated system while a human secretly or partially produces the response. A voice assistant prototype may route requests to a researcher. A travel service may have staff manually assemble recommendations behind a polished interface. This allows a team to test usefulness and interaction before building expensive automation.

The method can be ethical in a bounded research setting. Participants can consent to a study without being told every hypothesis in advance, if withholding detail is necessary, risk is low, data is protected, and debriefing follows. The design team learns whether the service deserves further investment. The illusion is temporary and governed.

The same setup becomes troubling when customers pay for "AI," share sensitive information under false assumptions, or depend on a service whose staffing cannot scale. Deception about capability changes expectations of privacy, speed, availability, and responsibility. A human operator may read material that the user thought was processed only by a machine.

Four questions for an ethical simulation

First, necessity: could the same learning be obtained without deception? A clearly labelled concierge prototype may answer the demand question. Second, exposure: what harm could occur if the simulated service makes a mistake or a person sees the data? Third, duration: is this a short test with a decision date, or an indefinite business model? Fourth, disclosure: who knows the truth now, and who will be told later?

Consent is not all or nothing. Researchers sometimes cannot reveal the exact hidden mechanism before a test because that knowledge would change behaviour. They can still state that the service is experimental, explain data handling,

avoid high-stakes tasks, and debrief participants. Commercial customers deserve a higher level of clarity because the company is not merely observing them; it is making a claim in exchange for money or trust.

Debriefing should do real repair. It should identify the concealed mechanism, explain why concealment was necessary, let participants withdraw data when feasible, and provide a contact for questions or harm. If the team would be embarrassed to give that explanation, the test design is warning them before launch. Ethical review is not paperwork added after invention; it is another prototype test for the social system around the product.

What the prototype is actually proving

A human behind the interface can prove that users value the outcome and help designers learn the right conversational pattern. It does not prove that a machine can produce the outcome accurately, cheaply, or at scale. Those are separate technical and economic hypotheses. Teams make a category error when success of the human-supported prototype is advertised as success of automation.

The difference resembles a film set. A painted facade can prove that a camera angle creates the desired image without proving that the building would stand. Everyone on the production understands the purpose. The ethical problem arises when an investor, buyer, or user is invited to infer structural reality from the facade.

Trust is part of the product

A service can eventually achieve real automation and still carry damage from its earlier claim. Users may reasonably ask what else was hidden, whether their data reached unexpected people, and whether present capability is independently verified. “Fake it until you make it” treats arrival as erasing the route. Trust remembers the route.

This does not require banning every illusion. It requires naming the boundary between a prototype and a public promise. Useful illusion is a research technique with limits, records, protection, and an exit. A hoax makes belief itself the product.

For transfer, imagine a school testing an automated tutoring service with teachers secretly selecting responses. The arrangement can reveal which explanations students request and where a conversation stalls. It cannot establish that a model gives accurate independent advice, and it raises special privacy and power concerns because students may not freely refuse. Label the service as experimental, use invented or low-risk content, disclose human review, set a short test period, and compare the manual evidence with a separately validated technical system before claiming arrival.

> **Sources for this chapter:** the official history of the Mechanical Turk (L093) and the service-design account of Wizard of Oz methods (L094). The chapter distinguishes a governed test from a commercial deception by purpose, risk, duration, and disclosure.

Chapter 5 - Demos, hidden labour, and the performance of readiness

A demonstration is a story about a product. The presenter chooses the conditions, sequence, camera, examples, and moment of success. That selection is unavoidable. A fair demo controls variables so a capability can be understood. A rigged demo controls them so viewers infer a capability the system does not have.

The dancer inside the robot announcement

At Tesla's 2021 AI event, Elon Musk introduced the idea of a humanoid "Tesla Bot." Before any robot appeared, a person in a white bodysuit danced on stage. The official curriculum links the costume reveal (L095). The absurdity was visible rather than secret, so it was not a Mechanical Turk style hoax. It was theatrical framing for a promised product. The risk was that spectacle and a confident timeline could make concept art feel closer to engineering reality than it was.

Later Optimus prototypes became physical machines, which is genuine progress. Public appearances still raised questions about autonomy and human assistance. The curriculum's link on a robot "still not quite there" (L096) discusses teleoperation, while reporting around the 2024 We, Robot event found that human operators assisted interactions. Teleoperation is a legitimate robotics tool. It supports data collection, training, remote work, and safe testing. The misleading step would be allowing an audience to believe assisted behaviour was fully autonomous.

A good demo label can preserve both excitement and truth: locomotion is autonomous, dialogue is remotely assisted, the route is pre-mapped, or the task succeeds in a controlled environment. Those details do not ruin the achievement. They locate it.

Rigging has degrees

The official gallery of staged technology demos (L097) includes cases that should not all receive one verdict. A pre-recorded backup video used after a live failure is different from footage presented as live. A product tested on a carefully prepared route is different from a computer animation presented without disclosure. A human selecting from suggested responses is different from a robot generating them independently.

Evaluate materiality, not just the word "rigged." Which component was real? What human assistance occurred? Were conditions representative of the claim? Did the presenter disclose the limitation before viewers made a decision? An engineering demo may appropriately isolate one subsystem. An investor presentation that implies the whole product works invites a broader inference and therefore owes broader evidence.

Amazon's Just Walk Out case needs a correction, not a slogan

Reports about Amazon's Just Walk Out checkout technology stated that about a thousand workers in India watched store video and helped determine purchases. The curriculum links the Ars Technica report (L098). That source is important because it made hidden human review visible, but scholars should also read the later Associated Press clarification.

Amazon disputed the claim that human reviewers were secretly acting as cashiers for every transaction. It said computer-vision models performed the checkout and human reviewers annotated video to improve the system and reviewed a minority of transactions. Human labelling and exception review are normal parts of many machine-learning systems. The serious questions remain: how large was the human role at each stage, what did customers understand about video processing, where and under what conditions was review work performed, and how did that role change over time?

Later evidence also prevents an outdated conclusion that the technology simply disappeared. In a September 2024 technical account, Amazon described a new multimodal receipt model and said Just Walk Out was operating in more than 180 third-party locations. That is Amazon's own dated account, so it should be read alongside independent

reporting rather than treated as a neutral audit. It nevertheless shows why removal from many Amazon Fresh stores and continuation in other venues can both be true.

The accurate lesson is stronger than the viral one. “AI” often describes a sociotechnical system containing models, sensors, cloud infrastructure, remote reviewers, store staff, and support teams. The presence of people does not prove the AI is fake. Concealing material human dependence can still mislead customers and investors about cost, privacy, scale, and autonomy.

Fireflies and the human notes behind an AI promise

The curriculum’s founder account about Fireflies.ai (L099) describes an early service sold as AI meeting assistance while the founders manually joined or listened to meetings and produced notes. A later report highlights the privacy and deception concerns.

As a demand test, manual note production could reveal what summaries users value before expensive automation exists. As a paid service entering private meetings under an AI identity, it changes informed consent. Participants may speak differently if they know an unknown person can hear the conversation. Employers may have confidentiality duties. The founders’ later technical success does not retroactively give early participants the disclosure they lacked.

A disclosure ladder for prototypes and demos

At the lowest-risk level, an internal team uses a fake door or human operator with no external data. Basic documentation may be enough. In a recruited study, participants should know they are testing an experimental service, understand data handling, and receive a debrief if the mechanism was concealed. In a public beta, limitations, human review, and support expectations should be stated before use. In a commercial or high-stakes service, material dependence, privacy exposure, reliability, and responsibility require clear disclosure and stronger validation.

Companies sometimes fear that honesty will make a demo less impressive. That is precisely the test of trust. If a capability has value only when assistance is hidden, the audience is being asked to purchase an inference rather than evaluate an achievement.

How to watch a demonstration like a scholar

Write down the exact claim before the demo begins. Observe whether the environment is controlled, whether inputs were selected in advance, and whether a person may be intervening. Separate appearance, movement, decision-making, conversation, and endurance into different capabilities. Ask what happens outside the prepared route and how many trials failed before the successful clip.

Then look for dates. A 2021 concept, a 2024 assisted demonstration, and a 2026 product should not be collapsed into one permanent verdict. Progress may be real while the earlier presentation remains misleading. The year theme asks whether we have arrived; a dated evidence trail shows which “there” was claimed at each point.

> **Sources for this chapter:** all five official demo and hidden-labour links: the 2021 Tesla Bot costume (L095), the later teleoperation concern (L096), the demo-history collection (L097), reporting on Just Walk Out (L098), and the Fireflies founder account (L099). The AP clarification qualifies the popular Amazon claim; Amazon's September 2024 technical account documents its later model and deployment claim from the company's own perspective.

Final synthesis - A prototype earns trust by revealing its boundary

The section begins with honest roughness. A sketch, paper screen, or exposed development board does not claim to be complete. Its unfinished form helps observers focus on the question being tested. As the guide moves toward public software and product demonstrations, roughness becomes easier to hide. A polished interface can conceal fragile code, a robot can conceal remote control, and an AI service can conceal human labour.

That does not make simulation the enemy. Wizard of Oz testing, teleoperation, manual concierge work, and controlled demos can reduce waste and teach teams what to build. Their legitimacy depends on scope and disclosure. The maker must know which hypothesis was tested. Participants must be protected. Viewers must not be encouraged to infer autonomy, safety, scale, or privacy that the evidence does not establish.

When a new case appears in an official round, ask four questions. What part is real? What part is represented or human-assisted? Who knows the difference? What evidence is still required before wider release? A prototype is not a promise that we are there. It is a disciplined way to discover what “there” would demand.